POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

EICTA Second Project Delivery - Neo4j and BPMN

Author(s): **Matteo Bregola (11095785)**

Emily Tiphareth Arcos (11021351)

Dario Antonino Santo (10733108)

Caterina Giulia Scarpetta (11095784)

Group Number: **5**

Academic Year: 2024-2025

Contents

Contents **i**

1 Neo4j **1**

1.1 Database Schema 1

1.2 Database Schema details 2

1.3 Database Schema Considerations 3

1.4 Neo4j Queries 5

1.4.1 CREATE/UPDATE/DELETE queries 5

1.4.2 MATCH statements and conditions 8

1.4.3 MATCH statement, conditions and aggregation without a WITH statement 11

1.4.4 MATCH statement, conditions and a WITH statement 13

1.4.5 MATCH statement, conditions and multiple WITH statements . . . 16

1.4.6 1 query with the shortestPath(...) function 20

2 BPMN **21**

2.1 Provided Process 21

2.2 BPMN Diagram 21

2.3 BPMN Diagram Considerations and Assumptions 25

1 | Neo4j

1.1. Database Schema

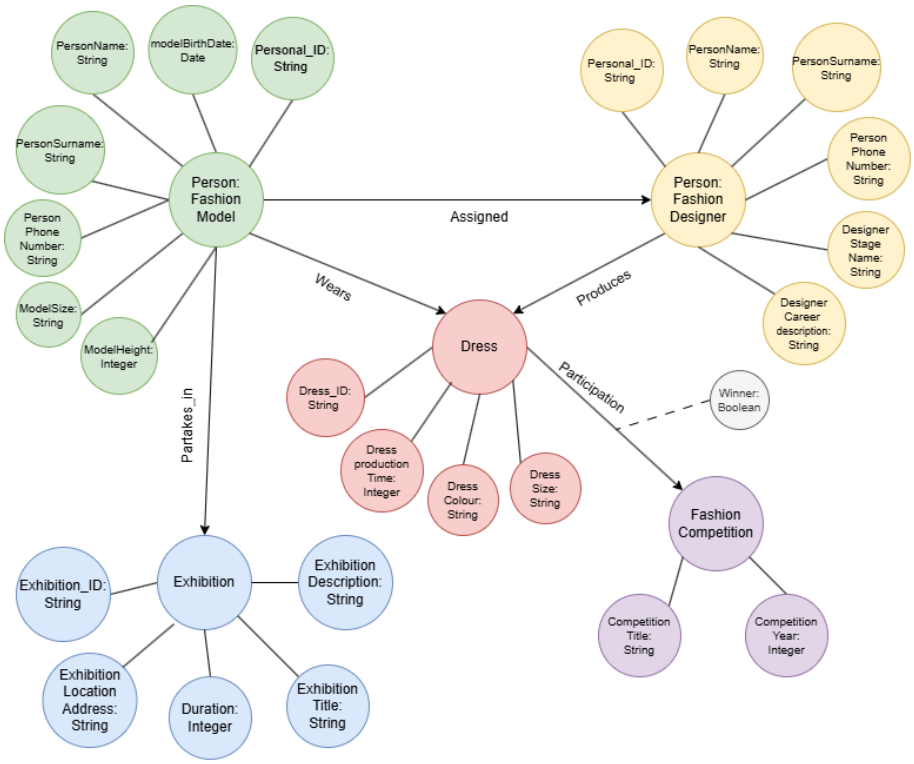

Figure 1.1: Neo4j schematic representation

1.2. Database Schema details

Entities: The entities we converted into nodes in our Neo4j database include:

- Fashion Model
- Fashion Designer
- Dress
- Exhibition
- Fashion Competition

Attributes: Next, following Project 1, we associated attributes to the entities identified based on the text descriptions below:

- Fashion Model: "Each model provides their personal data, including Personal ID, name, surname, birth date, phone number, height, and dress size (XS, S, M, L, XL, etc.)"
- Fashion Designer: "described by their Personal ID, name, surname, stage name , phone number, and a brief description of their career"
- Dress: "identified by a unique ID, size (XS, S, M, L, XL, etc.), colour, and production time"
- Fashion Exhibition: "described by a unique ID, duration (in minutes), title, location address, and description"
- Fashion Competition: "Once a year, a fashion competition is held" This entity did not have any direct listed attributes, but given this sentence in the text we decided to give this entity a title, as it would might be relevant.
- "Winner" in relationship "participation": the participation of a dress in a fashion competition came with the attribute "winner" to describe whether the dress had won the competition or not. This attribute was kept assigned to the relationship in Neo4j as it connects each dress to the competition they participate in.

The attributes have been listed in the database schematic representation and their types were chosen to follow from the previous SQL database.

Relationships:
Once again, the following relationships were carried on from Project 1:

- Assigned: "Models are assigned to 1 fashion designer". *Relationship direction:* set up to point from a model to a designer to show they are assigned to that designer - same flow as this phrase from the text.
- Produces: "Fashion designers produce dresses". *Relationship direction:* from the designer to the dress as suggested by the phrase structure.
- Partakes in: "Models partake in exhibitions". *Relationship direction:* from model to the exhibition as suggested by the phrase structure.
- Wears: "Models wear dresses". *Relationship direction:* from model to the dresses as suggested by the phrase structure.
- Participation: "The competition involves fashion designers submitting one of their dresses". *Relationship direction:* from dress to the competitions as the dress is submitted to each competition and therefore participates in it.

1.3. Database Schema Considerations

The passage from the previous ER-Diagram to Neo4j required two points of analysis:

- **How to map the "Person" entity**. In the SQL database it was reasonable to have three tables: one containing data shared by all types of people (model or designer) in our database, and two for each specific one, namely 'Fashion Model' and 'Fashion Designer'. In Neo4j, converting these tables directly would imply creating relationships between nodes of types *Person* and *Fashion Model* or *Fashion Designer*. This solution, albeit correct, doesn't seem efficient or useful as each person's data would be fragmented between nodes (also considering the dynamic nature of Neo4j, compared to SQL). Therefore, we decided to add the shared *Person*'s attributes back to *Fashion Model* and *Fashion Designer* types as was described in the original text. Furthermore, we added the *Person* label to all *Fashion Model* and *Fashion Designer* nodes. This approach maintains the idea of "Person" allowing retrieval of both models and designers by matching on the "Person" label.
- **Handling ID's**. The ER Model had several IDs (personal_ID, dress_ID, exhibition_ID, etc.) which were used to uniquely identify entities in the database. Neo4j automatically creates an <id>, unique for the specific entity type, for each node. Keeping the originally created IDs causes some nodes to have two unique identifiers, however, the original text states that "Each model provides their personal data, **including Personal ID**..." from which we may assume that the specific personal_ID

has more value for the user than a random unique identifier (*In the same way as we wouldn't want to substitute a SSN to a random <id>)*. Since the same reasoning can be done for the other entities, we decided to keep all the IDs coming from the ER model.

1.4. Neo4j Queries

In this section are reported the requested queries each one with its own title, code, description, and result. The results have been collected by using the provided dump and running the queries that modify the data (paragraph 1.4.1) as last operation to avoid including the changes in the output.

1.4.1. CREATE/UPDATE/DELETE queries

01. Create a "produces" relation between Fashion designer Vera Wang and a Dress with ID 187920

```
MATCH (f:Fashiondesigner), (d:Dress)
WHERE f.designerStageName = "Vera Wang" AND d.dressId = "187920"
CREATE (d)<-[r:produces]-(f)
RETURN r;
```

Query Description:
The query performs two match operations, one selecting nodes with the label "Fashiondesigner" and one with the label "Dress", the Fashiondesigner node has as stage Name "Vera Wang" while the Dress node have dressID= "187920". Then it creates a relationship labeled as "produces" from the designer ones to the dress nodes. Finally, it returns the created relation.

Query Result:


```
{
  "identity": 1159686799652552926,
  "start": 222,
  "end": 128,
  "type": "produces",
  "properties": {},
  "elementId": "5:e682cf4e-3052-430d-a205-c92267fc2edf
     :1159686799652552926",
  "startNodeElementId": "4:e682cf4e-3052-430d-a205-
     c92267fc2edf:222",
  "endNodeElementId": "4:e682cf4e-3052-430d-a205-
```

```
    c92267fc2edf:128"
}
```

02. Create a "partakes_in" relationship to connect a model with first name "Ku" and the exhibition with ID 561746

```
MATCH (m:Fashionmodel), (e:Exhibition)
WHERE m.personName = "Ku" AND e.exhibitionId = "561746"
CREATE (m)-[r:partakes_in]->(e)
RETURN r;
```

Query Description:

The query performs two match operations, one selecting nodes with the label "FashionModel" and one with the label "Exhibition". The query then finds one node of FashionModel with m.personName = "Ku" specified in the WHERE clause and the other node of Exhibition with an ID equal to "561746" also specified in the WHERE clause. Then, thanks to the the CREATE statement it creates a relationship named "partakes_in" from the Fashion Model to the Exhibition. Finally, it returns the relationship.

Query Result:


```
{
  "identity": 1155179901490298918,
  "start": 38,
  "end": 13,
  "type": "partakes_in",
  "properties": {},
  "elementId": "5:e682cf4e-3052-430d-a205-c92267fc2edf
    :1155179901490298918",
  "startNodeElementId": "4:e682cf4e-3052-430d-a205-
    c92267fc2edf:38",
  "endNodeElementId": "4:e682cf4e-3052-430d-a205-
    c92267fc2edf:13"
}
```

03. Update the Model with personal ID "223350" to size “M”

```
MATCH (fm:Fashionmodel)
WHERE fm.personalId = "223350"
SET fm.modelSize = "M"
RETURN fm
```

Query Description:

The query performs an update operation by matching the Fashion Model table, and uses a WHERE statement to find the fashion model with Personal ID equal to “223350”. Then using the SET statement it modifies the “modelSize” attribute to a medium size “M”. Finally, it returns the updated fashion model node.

Query Result:


```
{
  "identity": 34,
  "labels": [
    "Fashionmodel",
    "Person"
  ],
  "properties": {
    "personName": "Ye",
    "personalId": "223350",
    "personSurname": "Potter",
    "personPhoneNumber": "+359 790 326 9807",
    "modelHeight": 159,
    "modelBirthDate": "1973-12-30",
    "modelSize": "M"
  },
  "elementId": "4:79e66359-91d6-49b8-8f29-e86ec805bdb7
     :34"
}
```

04. Delete all the fashion competitions with title "Oscars Red Carpet" and the relationships linked to those nodes

```
MATCH (n:FashionCompetition{competitionTitle:"Oscars Red Carpet
    "})
DETACH DELETE n;
```

Query Description:
The query selects the nodes with the label "FashionCompetiton" and attibute competitionTitle equal to "Oscars Red carpet" through a MATCH statement, deletes the relationship that involves these nodes (by using the detach) and then deletes the nodes.

Query Result:
Deleted 3 nodes, deleted 7 relationships.

05. Delete the relationship between the fashion model with surname Sweetenham and the exhibition they attended

```
MATCH (FashionModel{personSurname:"Swettenham"})-[a:partakes_in
    ]->()
DELETE a
```

Query Description:
The query selects a node of label "FashionModel" and who has attribute personSurname set to "Sweetenham", takes the existing "partakes_in" relationship the model has and deletes this relationship.

Query Result:
Deleted 1 relationship.

1.4.2. MATCH statements and conditions

01. Return all the models that partake in an exhibition that lasts more than 100 minutes and have a height less than 175 cm

```
MATCH (a:Fashionmodel) -[:partakes_in]-> (b:Exhibition)
WHERE b.duration > 100 AND a.modelHeight < 175
RETURN a
```

Query Description:

The query through the MATCH statement finds all the nodes with label "FashionModel" that are in a relationship "partakes_in" with "Exhibition" nodes. Then thanks to the WHERE clause it will retrieve only Exhibitions with a duration greater than 100 minutes and models with a height less than 175 cm. Both conditions are respected thanks to the AND operator. Finally with the RETURN statement the query will display all the models' nodes.

Query Result:

a
(:Fashionmodel:Person {personName: Ye, personalId: 223350, personSurname: Potter, personPhoneNumber: +359 790 326 9807, modelHeight: 159, modelBirthDate: 1973-12-30, modelSize: L})
(:Fashionmodel:Person {personName: Leonore, personalId: 486337, personSurname: Keegan, personPhoneNumber: +55 785 848 8863, modelHeight: 163, modelBirthDate: 1988-04-01, modelSize: XL})
(:Fashionmodel:Person {personName: Zoe, personalId: 701961, personSurname: Swettenham, personPhoneNumber: +63 586 448 4840, modelHeight: 168, modelBirthDate: 1998-02-08, modelSize: S})
(:Fashionmodel:Person {personName: Grazia, personalId: 406939, personSurname: Sanney, personPhoneNumber: +7 723 655 3579, modelHeight: 171, modelBirthDate: 1990-02-15, modelSize: L})
(:Fashionmodel:Person {personName: Kui, personalId: 158959, personSurname: Habens, personPhoneNumber: +86 201 870 0528, modelHeight: 172, modelBirthDate: 1991-02-02, modelSize: M})
(:Fashionmodel:Person {personName: Mahelie, personalId: 191180, personSurname: Nutting, personPhoneNumber: +86 175 117 8060, modelHeight: 151, modelBirthDate: 1976-09-27, modelSize: XS})
(:Fashionmodel:Person {personName: Zhi, personalId: 561919, personSurname: Scragg, personPhoneNumber: +33 282 197 8699, modelHeight: 156, modelBirthDate: 1986-11-07, modelSize: 2XL})
(:Fashionmodel:Person {personName: Solene, personalId: 866930, personSurname: Blesdill, personPhoneNumber: +7 548 369 4088, modelHeight: 167, modelBirthDate: 2002-08-29, modelSize: S})
(:Fashionmodel:Person {personName: Lai, personalId: 796055, personSurname: Botterill, personPhoneNumber: +33 627 693 0473, modelHeight: 152, modelBirthDate: 1990-01-29, modelSize: L})

Table 1.1: Result from Query 1.4.2.1

02. Return the first and last name of all fashion models wearing a Violet dress of size M alongside their dress IDs

```
MATCH (fm:Fashionmodel)-[:wears]->(d:Dress)
WHERE d.dressColour = "Violet" AND d.dressSize = "M"
RETURN fm.personName, fm.personSurname, d.dressId
```

Query Description:
This query matches fashion models along with the "wears" relationship to dresses. It then filters the results using a WHERE clause to find dresses that are both violet in color and of size "M". Finally, it returns the first name, last name, and dress ID of the fashion models who meet these criteria.

Query Result:

fm.personName	fm.personSurname	d.dressId
Ku	Dadson	242985
Kalliste	Gilhooly	357420

Table 1.2: Result from Query 1.4.2.2

03. Retrieve the titles of exhibitions that featured a dress with over 100 minutes of production time and that also participated in the "SAG Awards Red Carpet" fashion competition

```
MATCH(fc:FashionCompetition)<-[p:participation]-(d:Dress)
<-[w:wears]-()-[pi:partakes_in]->(e:Exhibition)
WHERE fc.competitionTitle="SAG Awards Red Carpet" AND
d.dressProductionTime>100
RETURN e.exhibitionTitle
```

Query Description:
The match statement searches for a pattern composed by 4 nodes and 3 relationships. Nodes labels:

- Fashion Competition
- Dress
- any label
- Exhibition

Relationships:

- participation, from Dress to FashionCompetition
- wears, from the unnamed node to Dress
- partakes_in, from the unnamed node to exhibition

After having found a match for this pattern, the WHERE clause filters based on two conditions: the competitionTitle of the FashionCompetition node must be "SAG Awards Red Carpet" and the dressProductionTime of the Dress node must be greater than 100. Finally, it returns the exhibition title.

Query Result:

e.exhibitionTitle
Vienna Fashion Week
Shanghai Fashion Week

Table 1.3: Result from Query 1.4.2.3

1.4.3. MATCH statement, conditions and aggregation without a WITH statement

01. Return the earliest year that a dress made in over 2 hours from Tom Ford participated in a fashion competition

```
MATCH (fd:Fashiondesigner)-[:produces]->(d)
-[:participation]->(competition)
WHERE fd.designerStageName = "Tom Ford" AND d.
    dressProductionTime>120
RETURN min(competition.competitionYear)
```

Query Description:
Through the MATCH statement, the query connects the fashion designer node to the fashion competition through the 'produces' and 'participation' relationships passing through the dress node 'd'. The WHERE statement filters the results to values where the fashion designer stage name is equal to 'Tom Ford' AND the dress production time is over 120 min. The RETURN statement finds the earliest competition where these constraints are met using the MIN aggregation function.

Query Result:

min(competition.competitionYear)
1981

Table 1.4: Result from Query 1.4.3.1

02. Return the year gap between the newest and oldest dress of size S or XS made by Christian Dior that participated in a competition

```
MATCH(fd:Fashiondesigner{designerStageName:"Christian Dior"})
-[p:produces]->(d:Dress)-[p2:participation]->(fc:
    FashionCompetition)
WHERE d.dressSize="XS" OR d.dressSize="S"
RETURN(max(fc.competitionYear)-min(fc.competitionYear)) AS Gap
```

Query Description:
The match statements search for a pattern in which there are three nodes connected by two relationships. The "first" node of the pattern has fashion designer as label with "Christian Dior" as a stage name, this node has a "produces" relationship with a dress which in turn has a relationship of type "participation" with fashionCompetition. The pattern resulting from the match is then filtered based on the dress size, keeping only dresses of size XS or S. Finally, performs subtraction between the maximum and minimum values of the competition year returning this value inside a column called "Gap".

Query Result:

Gap
1

Table 1.5: Result from Query 1.4.3.2

1.4.4. MATCH statement, conditions and a WITH statement

01. Return all Fashion designers who work at least with 2 models who partake in exhibitions that last more than 60 minutes

```
MATCH (a:Fashiondesigner)<-[:assigned]-(b:Fashionmodel)
-[:partakes_in]-> (c:Exhibition)
WHERE c.duration > 60
WITH a, COUNT(b) AS model_count
WHERE model_count >= 2
RETURN a
```

Query Description:
The MATCH statement, thanks to the nodes b, finds all the indirect relationships between Fashiondesigner and Exhibition. Then through the first WHERE clause it will retrieve only the exhibition that lasts more than 60 minutes. The WITH statement and the COUNT operator allow to find all the models assigned to nodes a. The AS operator allows to simplify the naming process. The second WHERE clause retrieves designers who had at least 2 models assigned. Then the RETURN statement returns all the fashion designer's nodes with all the conditions above.

Query Result:

a
(:Fashiondesigner:Person {personName: Sophie, personalId: 844141, personSurname: Cabbell, personPhoneNumber: +81 625 236 7635, designerStageName: Diane von Furstenberg, designerCareerDescription: sleek and futuristic})
(:Fashiondesigner:Person {personName: Leyla, personalId: 489100, personSurname: Wishart, personPhoneNumber: +63 484 895 5372, designerStageName: Thom Browne, designerCareerDescription: athletic and functional})
(:Fashiondesigner:Person {personName: Suzi, personalId: 792278, personSurname: Fernihough, personPhoneNumber: +257 953 506 5552, designerStageName: Donatella Versace, designerCareerDescription: sporty and casual})
(:Fashiondesigner:Person {personName: Hobey, personalId: 777697, personSurname: Thornley, personPhoneNumber: +62 492 134 1485, designerStageName: Tom Ford, designerCareerDescription: sporty and casual})
(:Fashiondesigner:Person {personName: Serge, personalId: 188385, personSurname: Darkin, personPhoneNumber: +86 771 344 0681, designerStageName: Jeremy Scott, designerCareerDescription: playful and whimsical})
(:Fashiondesigner:Person {personName: Armin, personalId: 525437, personSurname: Gircke, personPhoneNumber: +86 289 767 3782, designerStageName: Calvin Klein, designerCareerDescription: minimalist and monochromatic})
(:Fashiondesigner:Person {personName: Anet, personalId: 898125, personSurname: Mostin, personPhoneNumber: +48 644 950 7799, designerStageName: Vera Wang, designerCareerDescription: timeless and classic})
(:Fashiondesigner:Person {personName: Wood, personalId: 732291, personSurname: Guerrieri, personPhoneNumber: +255 858 259 6025, designerStageName: Stella McCartney, designerCareerDescription: ethereal and dreamy})
(:Fashiondesigner:Person {personName: Tania, personalId: 246042, personSurname: Pecht, personPhoneNumber: +231 609 520 7728, designerStageName: Zac Posen, designerCareerDescription: urban and streetwear})
(:Fashiondesigner:Person {personName: Humbert, personalId: 704908, personSurname: Sargison, personPhoneNumber: +48 288 790 7498, designerStageName: Phillip Lim, designerCareerDescription: ethereal and dreamy})

Table 1.6: Result from Query 1.4.4.1

02. Find three Fashion designers which have won all of the competitions they submitted dresses for

```
MATCH (fd:Fashiondesigner)-[:produces]->(d)-[pa:participation
    ]->()
WITH fd, count(d) AS dressesInComps, count(pa.winner = TRUE) AS
    winningDresses
WHERE dressesInComps = winningDresses
LIMIT 3
RETURN fd
```

Query Description:

The MATCH statement matches fashion designers which have produced dresses (connected with the 'produces' relationship) that have participated in fashion competitions through the 'participation' relationship. The WITH statement then counts the amount of dresses each fashion designer has submitted to competitions denoted 'dressesInComps' using the COUNT function, and also sums per designer the amount of dresses that won competitions denoted 'winningDresses' using the SUM function. The WHERE statement then filters the data only where the amount of dresses submitted is equal to the amount of wins. The LIMIT function then ensures that only three designers are returned. Finally, the query returns the fashion designer nodes which satisfy the condition.

Query Result:

fd
(:Fashiondesigner:Person {personName: Nadine, personalId: 149103, personSurname: Havelin, personPhoneNumber: +55 120 328 4679, designerStageName: Giorgio Armani, designerCareerDescription: artisanal and handcrafted})
(:Fashiondesigner:Person {personName: Serge, personalId: 188385, personSurname: Darkin, personPhoneNumber: +86 771 344 0681, designerStageName: Jeremy Scott, designerCareerDescription: playful and whimsical})
(:Fashiondesigner:Person {personName: Tania, personalId: 246042, personSurname: Pecht, personPhoneNumber: +231 609 520 7728, designerStageName: Zac Posen, designerCareerDescription: urban and streetwear})

Table 1.7: Result from Query 1.4.4.2

03. Return designers stage name and number of dresses of fashion designers which have produced more than 2 dresses

```
MATCH (d:Dress)<-[p:produces]-(f:Fashiondesigner)
WITH f, COUNT(p) AS dressCount
WHERE dressCount > 2
RETURN f.designerStageName, dressCount;
```

Query Description:

The match statement matches pairs of fashion designers' nodes and their associated dress nodes via the produces relationship. The With clause is adopted to "use" the fashion designer in the next part of the query and to aggregate results. The count, stored in a variable name "dressCount", enumerates the "produces" relationship for each designer. Finally, the designer's stage name and the total number of dresses produced by him/her are returned.

Query Result:

f.designerStageName	dressCount
Zac Posen	3
Stella McCartney	3
Tom Ford	4
Diane von Furstenberg	5

Table 1.8: Result from Query 1.4.4.3

1.4.5. MATCH statement, conditions and multiple WITH statements

01. Retrieve the Count of Fashion Models Wearing Size 'M' Dresses and Participating in Exhibitions Longer Than 30 Minutes

```
MATCH (fm:Fashionmodel)-[:wears]->(d:Dress)<-[:produces]-(fd:
    Fashiondesigner)
WHERE d.dressSize = 'M'
WITH fm, d, fd
MATCH (fm)-[:partakes_in]->(ex:Exhibition)
WHERE ex.duration > 30
WITH d, fd, ex, count(fm) AS ModelCount
RETURN d.dressSize AS dress_size, fd.designerStageName AS
    designer_name,
        ex.exhibitionTitle AS exhibition_title, ex.
            exhibitionLocationAddress AS exhibition_location,
            ModelCount
```

Query Description:

This query matches several nodes: fashion models, dresses, and fashion designers in the first part. It identifies fashion models who wear dresses that were produced by fashion designers. The query then filters for dresses of size 'M' using the WHERE clause. Next, the WITH statement passes the information (fashion models, dresses, and designers) to the next part of the query. Here, we match the fashion models who participate in exhibitions. We further filter exhibitions that last more than 30 minutes. Using another WITH statement, we group the results by dress, fashion designer, and exhibition, while counting how many models (fm) participated in each exhibition. This count is stored as ModelCount. Finally, the query returns the following information:

- Designer (Stage) Name
- Exhibition Title
- Exhibition Location
- Model Count (for each exhibition)

Query Result:

Dress Size	Designer Name	Exhibition Title	Exhibition Location	Model Count
M	Tom Ford	Paris Fashion Week	789 Oak Rd	1
M	Oscar de la Renta	Berlin Fashion Week	Somewhere City	1
M	Valentino Garavani	Berlin Fashion Week	Somewhere City	1
M	Calvin Klein	Miami Fashion Week	Everywhere Village	1
M	Zac Posen	Toronto Fashion Week	USA	1

Table 1.9: Result from Query 1.4.5.1

02. Retrieve the designer's name and the number of models assigned to them if the designer is assigned more than one model and has won a fashion competition before 2010

```
MATCH(fc:FashionCompetition)<-[p:participation]-(d:Dress)
<-[produces]-(fd:FashionDesigner)
WHERE fc.competitionYear < 2010 AND p.winner=true
WITH fd
MATCH (fd)<-[a:assigned]-(fm:FashionModel)
WITH fd, COUNT(fm) AS models_assigned
WHERE models_assigned > 1
RETURN fd.designerStageName, models_assigned
```

Query Description:
The query is designed in two parts: the first subquery finds fashion designers who have won a fashion competition before 2010, the second subquery retrieves the models assigned to these filtered designers.

The first subquery matches a pattern composed of 3 nodes and 2 relationships. The nodes involved are Fashion Competition, Dress and Fashion Designers. The relationships are:

- participation: from Dress to Fashion Competition
- produces: from Fashion Designer to Dress

Then, the where clause filters the results based on the competition year and participation winner attribute that should be respectively less than 2010 and true. The with passes forward only the Fashiondesigner nodes that meet these conditions.

The second subquery searches a pattern made by fashion designers nodes (retrieved thanks to the with clause) that have an assigned relationship with a fashion model. Then it counts how many Fashionmodel nodes are assigned to each designer, with the WITH

clause passing fd and using "AS" to rename the label. Finally, the RETURN statement outputs the designerStageName of each designer and the count of models assigned to them.

Query Result:

fd.designerStageName	models_assigned
Vera Wang	2
Donatella Versace	2
Raf Simons	2
Oscar de la Renta	2
Giorgio Armani	2
Coco Chanel	2
Tom Ford	4
Calvin Klein	2
Jeremy Scott	2
Valentino Garavani	2
Isabel Marant	2
Diane von Furstenberg	5
Christian Dior	2
Ralph Lauren	2
Thom Browne	2

Table 1.10: Result from Query 1.4.5.2

1.4.6. 1 query with the shortestPath(...) function

Returns the shortest path between Fashion Designers and Fashion Competitions held in 2000s

```
MATCH a = shortestPath((b:Fashiondesigner)-[*]->(c:
    FashionCompetition))
WHERE c.competitionYear >= 2000
RETURN a LIMIT 3
```

Query Description:
The MATCH clause retrieves the nodes from Fashion Designer and Fashion competition which are connected. The WHERE clause filters all Fashion Competitions held in the 2000s. The function shortestPath allows to find the shortest path between Fashion Designer and Fashion Competition which are not directly linked. The * operator is used to define distance in cypher. Finally the LIMIT statement allows to limit the result to three.

Query Result:

a
(:Fashiondesigner:Person {personName: Tania, personalId: 246042, personSurname: Pecht, personPhoneNumber: +231 609 520 7728, designerStageName: Zac Posen, designerCareerDescription: urban and streetwear})-[:produces]->(:Dress {dressColour: Khaki, dressSize: L, dressId: 883120, dressProductionTime: 156})-[:participation {winner: true}]->(:FashionCompetition {competitionTitle: Sundance Film Festival Fashion, competitionYear: 2012})
(:Fashiondesigner:Person {personName: Frederik, personalId: 454170, personSurname: Molden, personPhoneNumber: +63 361 516 2326, designerStageName: Betsey Johnson, designerCareerDescription: feminine and romantic})-[:produces]->(:Dress {dressColour: Purple, dressSize: XS, dressId: 835583, dressProductionTime: 125})-[:participation {winner: true}]->(:FashionCompetition {competitionTitle: SAG Awards Red Carpet, competitionYear: 2001})
(:Fashiondesigner:Person {personName: Leyla, personalId: 489100, personSurname: Wishart, personPhoneNumber: +63 484 895 5372, designerStageName: Thom Browne, designerCareerDescription: athletic and functional})-[:produces]->(:Dress {dressColour: Pink, dressSize: S, dressId: 343333, dressProductionTime: 92})-[:participation {winner: true}]->(:FashionCompetition {competitionTitle: Emmy Awards Red Carpet, competitionYear: 2006})

Table 1.11: Result from Query 1.4.6.1

2 | BPMN

2.1. Provided Process

Given the following process, write the BPMN diagram representing it.
Models submit their CVs through the company's front desk. As soon as a CV is received, it is forwarded to the HR department, which checks it. If they deem the candidate suitable for a position, they forward the CV to the head of the company. Otherwise, the candidate is noticed, and the process terminates. The head of the company contacts the fashion designers (internals) who work for the company and submits the model's CV to them. Designers have three days to reply to the head (a response is mandatory, even if negative). If a designer is interested in working with the model, the head asks the front desk to contact the model to set up a meeting with them. If no designer is interested, the candidate is notified, and the process terminates. If the model answers affirmatively, the designer is contacted through the front desk, and they are asked to prepare a dress for the model. The dress is sent to the model, and then a meeting with the head is held. After the meeting, the head of the company provides their final decision through the front desk. They have at most 5 days to do so. Otherwise, the candidate is notified, and the process terminates.

2.2. BPMN Diagram

For clarity, the BPMN diagram generated from the above text has been split into segmented images to ensure legibility - the full BPMN diagram will be accessible in the project delivery folder. To note, red asterisks * are used to track ongoing flows from the previous segment to track tasks between figures.

All considerations and assumptions made for the creation of the below diagram will be found in the next section "BPMN Diagram Considerations".

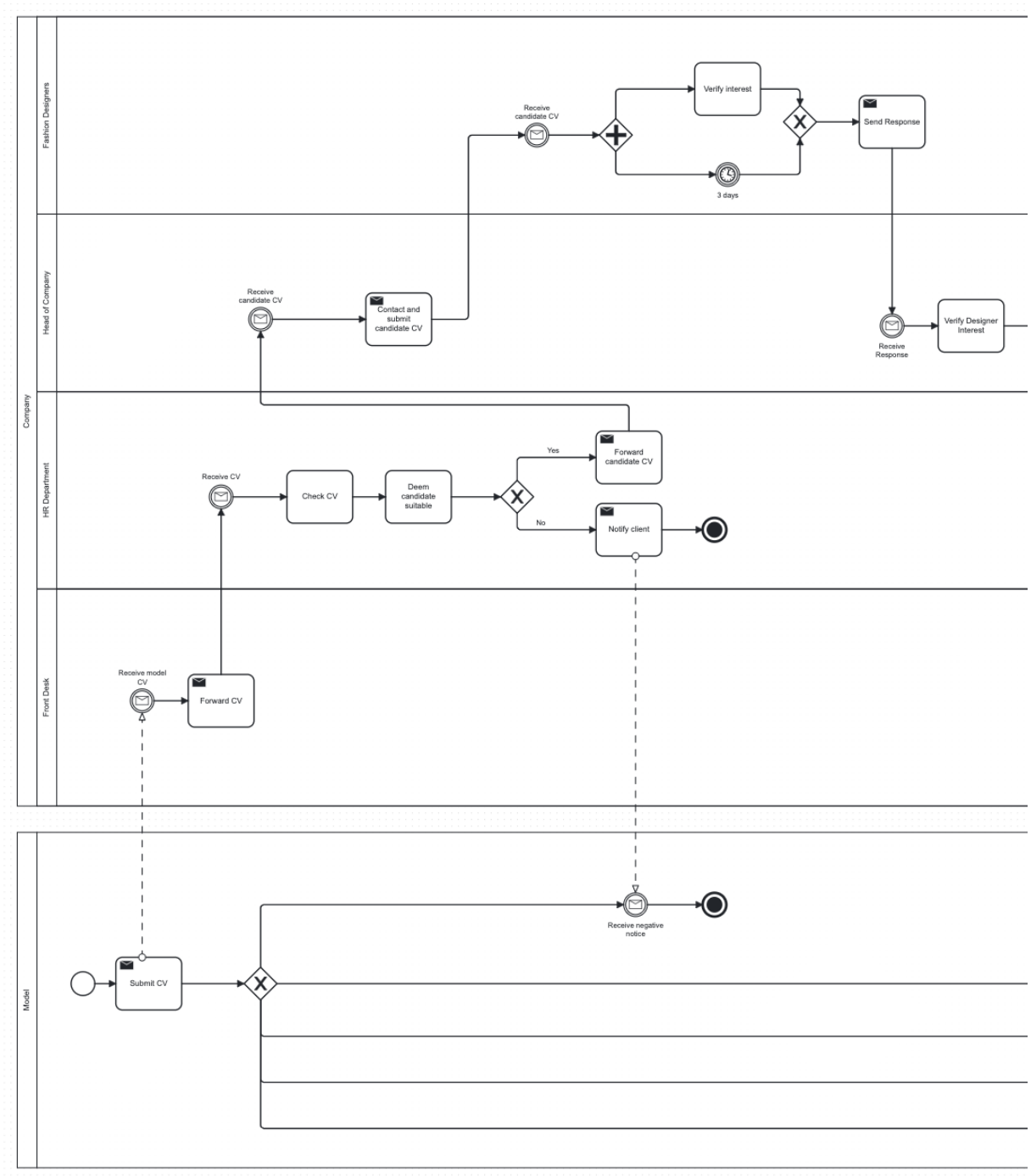

Figure 2.1: First segment of BPMN diagram

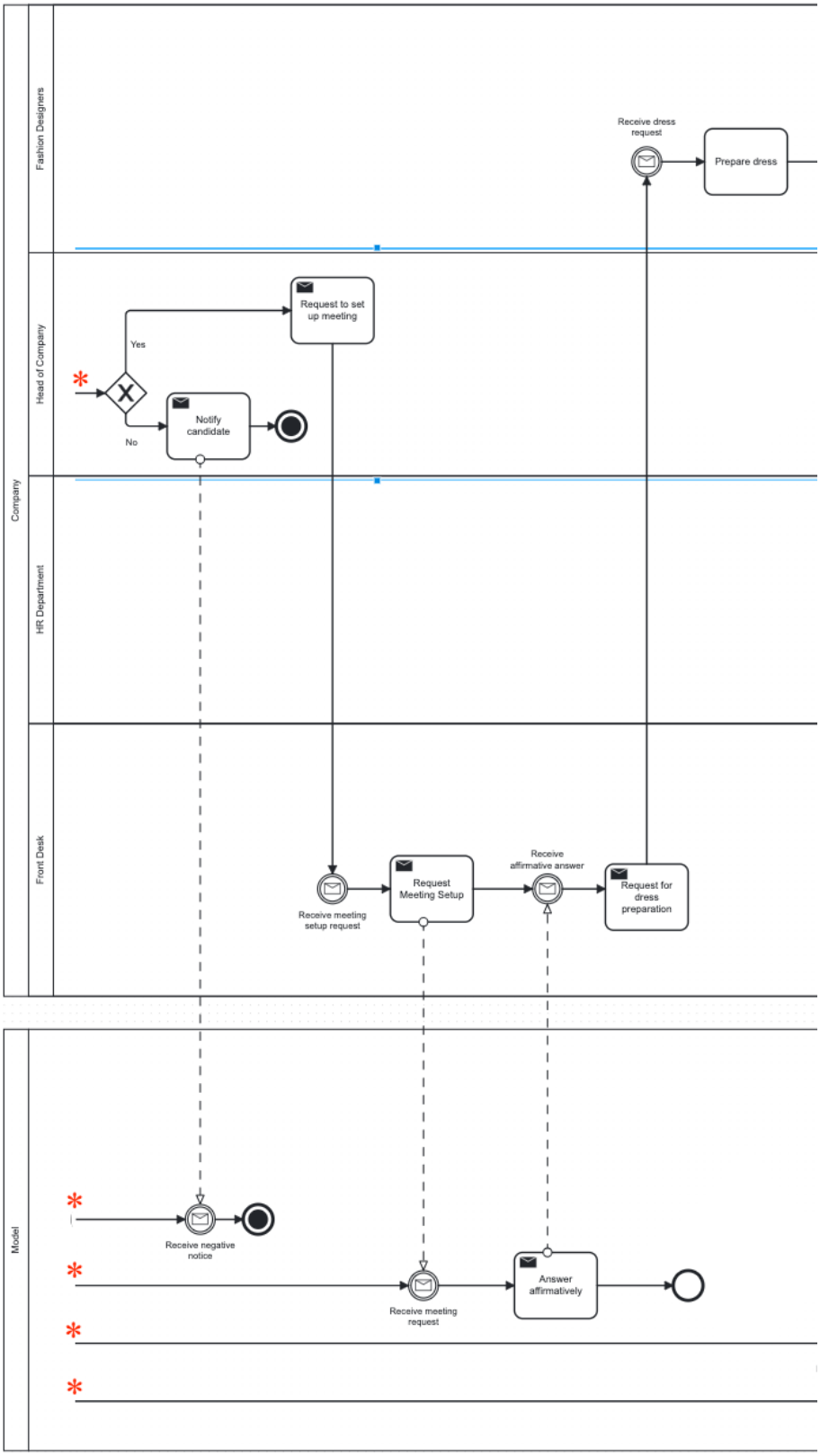

Figure 2.2: Second segment of BPMN diagram

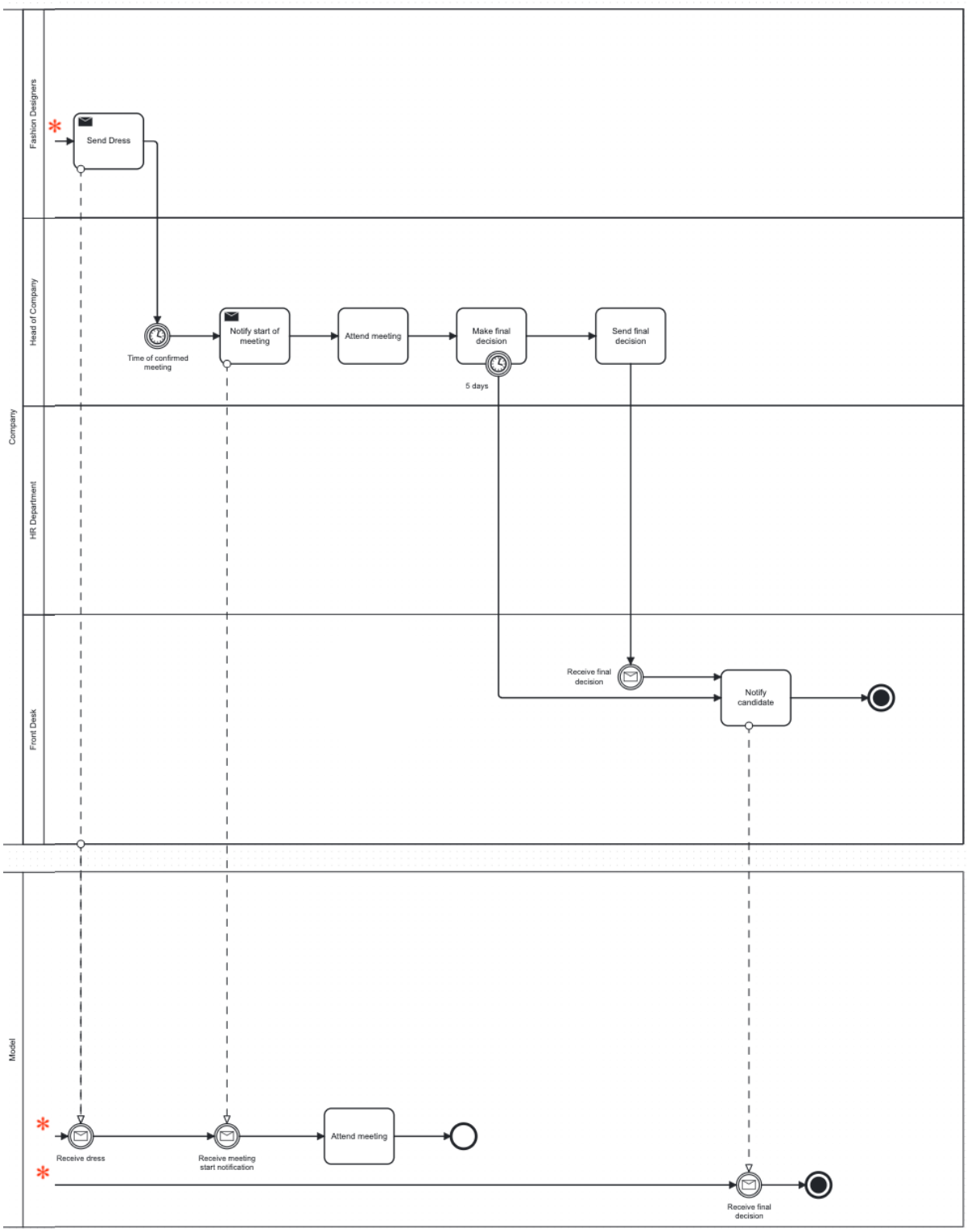

Figure 2.3: Third segment of BPMN diagram

2.3. BPMN Diagram Considerations and Assumptions

In this section, we will discuss parts of the provided text that led to specific decisions in the final output of the BPMN diagram.

1. The text describes the movement of a CV from the model candidates to a front desk, and then between members of the company for consideration. The flow to represent in the diagram is therefore of sending and receiving the file. We acknowledge the CV is a document and could be represented by a data object, however we believe this is implied by the send tasks we represented, and each send task is different in this case.

2. According to the phrase from the text "Designers have three days to reply to the head (a response is mandatory, even if negative)" we concluded that the process should allow for two options to occur: the designers can verify their interest in the model within three days, or after three days a response deadline will be triggered forcing an answer (Figure 2.1).

3. Within the text, a model is sent a meeting request and can respond affirmatively. Given the model applies to the company and triggers this process, we made the assumption that the model would not have an alternative to this response, as the circumstance presumes that they wish to be hired (Figure 2.2).

4. The text describes the following sequence of events: "If the model answers affirmatively, the designer is contacted through the front desk, and they are asked to prepare a dress for the model. The dress is sent to the model, and then a meeting with the head is held." From this, we have decided to make the following assumptions and decisions:

- the designers will be able to complete and send the dress before the scheduled meeting date and time
- given the dress is being sent by the designers to the model, we identified it as a send task which is received (similarly to a message) by the model
- the model agreed on a meeting date organised by the front desk for the head of the company and therefore the "Attend meeting" tasks in both model and head of company lanes are simultaneous. To achieve this, we have added a notification sent by the head of the company to alert the model of the start and trigger both parties

to attend.

5. The final phrase of the text states "They have at most 5 days to do so. Otherwise, the candidate is notified, and the process terminates." From this, we gathered that there were two options to follow in the flow: either the head of the company sends their final decision to the front desk to notify the model, or after 5 days the notify action passes directly to the front desk (Figure 2.3).